RANDOM FOREST CLASSIFICATION PROJECT REPORT

Project Title: Predictive Classification Using Random Forest Classifier

Student: Kumar Aditya Subham

Dataset: Loan Approval Prediction

Link: **https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset**

File: test_Y3wMUE5_7gLdaTN.csv

Introduction:

Random Forest is a supervised machine learning algorithm that combines multiple decision trees to make predictions. It is useful for classification problems because it can handle different types of features and learn complex patterns.

In this project, I used the Loan Prediction dataset from Kaggle to practise data preprocessing, visualization, model training, and evaluation. However, the available dataset did not contain the Loan_Status column. Therefore, I used Property_Area as the target variable to build a three-class classification model.

Objectives:

The main objectives of this project are:

1. To understand and analyse a real-world dataset.
2. To identify and handle missing values and duplicate records.
3. To perform exploratory data analysis using graphs and visualizations.
4. To handle outliers and convert categorical data into numerical form.
5. To implement a Random Forest Classification model.
6. To apply SMOTE to balance the training data.
7. To use GridSearchCV for hyperparameter tuning.
8. To evaluate the model using accuracy and classification metrics.

Evaluation Metrics

Metric	Description
Confusion Matrix	It shows the correct and incorrect predictions made by the model for Rural, Semiurban, and Urban areas using a heatmap.
Precision and Recall	Precision shows how many predictions are correct, while recall shows how many actual values are correctly predicted by the model.
F1-Score	It combines precision and recall into one score and helps us understand the overall performance of the model.
ROC-AUC Curve	It shows how well the model separates different classes by comparing the true positive rate and false positive rate.
Feature Importance	It shows which input features have the most influence on the predictions made by the Random Forest model.

Data Preprocessing

• Handling Missing Values

The dataset initially contained missing values in several columns. Missing categorical values, such as Gender, Dependents, and Self_Employed, were filled using the mode. Missing numerical values, such as LoanAmount, Loan_Amount_Term, and Credit_History, were filled using the median.

After this process, the dataset had no remaining missing values.

• Duplicate Checking

I checked the dataset for duplicate records. No duplicate rows were found in the version of the dataset used for this analysis.

• Exploratory Data Analysis

EDA was performed to understand the dataset and identify patterns in the data. Count plots were created for Property_Area and Married. I also compared Married with Property_Area and Gender with Married.

A correlation heatmap was used to understand the relationships between numerical variables. These visualizations helped me understand the distribution of the data before model training.

• Outlier Detection and Treatment

Box plots and the Interquartile Range (IQR) method were used to identify outliers in numerical columns.

Outliers were identified in ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, and Credit_History. IQR-based capping was applied to ApplicantIncome, CoapplicantIncome, LoanAmount, and Loan_Amount_Term. Credit_History was not capped.

• Categorical Encoding

Machine learning models require numerical input. Therefore, One-Hot Encoding was applied to convert categorical features into numerical columns. The Loan_ID column was removed because it is an identifier and does not provide meaningful predictive information.

Model Development

• Train-Test Split

The dataset was divided into training and testing sets using an 80:20 ratio. The `train_test_split()` function was used with `random_state=42` and stratification to maintain the class proportions.

The training data was used to train the model, while the testing data was used to evaluate predictions on unseen records.

• Handling Class Imbalance Using SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) was applied to the training data to balance the number of samples in each target class.
Before SMOTE, the training classes had different sample counts. After applying SMOTE, the training dataset contained 336 samples, with 112 samples for each class.
SMOTE was applied only to the training data to avoid directly modifying the test data.

• Random Forest Classifier

The Random Forest algorithm was selected because it combines multiple decision trees to make a final prediction. Each tree learns patterns from the training data, and the predictions are combined to determine the final class.

The model was trained to classify records into Rural, Semiurban, and Urban categories based on the available applicant-related features.

• Hyperparameter Tuning

GridSearchCV was used to find suitable hyperparameters for the Random Forest model. Five-fold cross-validation was used to compare different combinations of parameters.

The scoring method was changed to `f1_macro` because the target variable contains three classes.

The best parameters obtained were:

Parameter	Best value
Number of trees (n_estimators)	200
Maximum depth (max_depth)	10
Minimum samples to split (min_samples_split)	5
Minimum samples in a leaf (min_samples_leaf)	2
Best cross-validation F1-macro score	0.4805

These parameters were selected based on the cross-validation results. The tuned model was then evaluated using the training and testing datasets.

• Model Evaluation and Results

The tuned Random Forest model produced the following accuracy results:

Evaluation Metric	Result
Training Accuracy	92.56%
Testing Accuracy	32.43%
Best Cross-Validation F1-Macro Score	48.05%

The training accuracy is much higher than the testing accuracy. This suggests that the model may be overfitting and is not generalizing well to unseen data.

In the initial model evaluation, the testing accuracy was 29.73%. After hyperparameter tuning, it increased to 32.43%. Although there was a small improvement, the overall testing performance remained low.

The classification report for the initial model also showed low F1-scores for the three classes. This indicates that the model had difficulty correctly classifying Rural, Semiurban, and Urban records.

The confusion matrix, ROC-AUC analysis, and feature-importance chart should also be included in the final submission after generating them from the trained model.

Key Insights :

1. In this project, I used the Random Forest algorithm to predict whether a loan should be approved or rejected.

2. The model achieved 92.56% training accuracy, but the testing accuracy was only 32.43%.

3. The difference between training and testing accuracy shows that the model may be overfitting.

4. The best cross-validation F1-macro score was 48.05%, which means the model still needs improvement.

5. Hyperparameter tuning helped me find the best values for the Random Forest model.

6. Through this project, I learned about data preprocessing, model training, hyperparameter tuning, and model evaluation.

7. I understood that high training accuracy does not always mean that a model will perform well on new data.

Limitations

The project has the following limitations:

1. The dataset contains only 367 records, which limits the amount of data available for learning.
2. The testing accuracy is low, indicating poor prediction performance on unseen data.
3. The difference between training and testing accuracy suggests possible overfitting.
4. SMOTE can generate synthetic samples, but it does not guarantee better model performance.
5. The available dataset does not contain the `Loan_Status` column. Therefore, this model does not predict whether a loan is approved or rejected.

Conclusion

In this project, I implemented a Random Forest Classification model using a dataset obtained from Kaggle. I performed data cleaning, missing-value handling, exploratory data analysis, outlier treatment, categorical encoding, and class balancing using SMOTE. I also applied GridSearchCV to identify suitable hyperparameters for the model.

The final tuned model achieved 92.56% training accuracy and 32.43% testing accuracy. The results indicate that the model may be overfitting and requires further improvement. This project helped me understand the complete machine learning workflow, from preparing the dataset to training and evaluating a classification model.

Since the available dataset did not contain the `Loan_Status` column, the model predicts the property area category rather than loan approval. Using a dataset with the correct target variable and improving the model evaluation process would be important next steps.